

LIYEMA MNCORA

ST10397165

PROGRAMMING 2B

PROG6212/w

POE PART 1

**Bachelor of Computer and Information Sciences
in Application Development**

BCAD0701

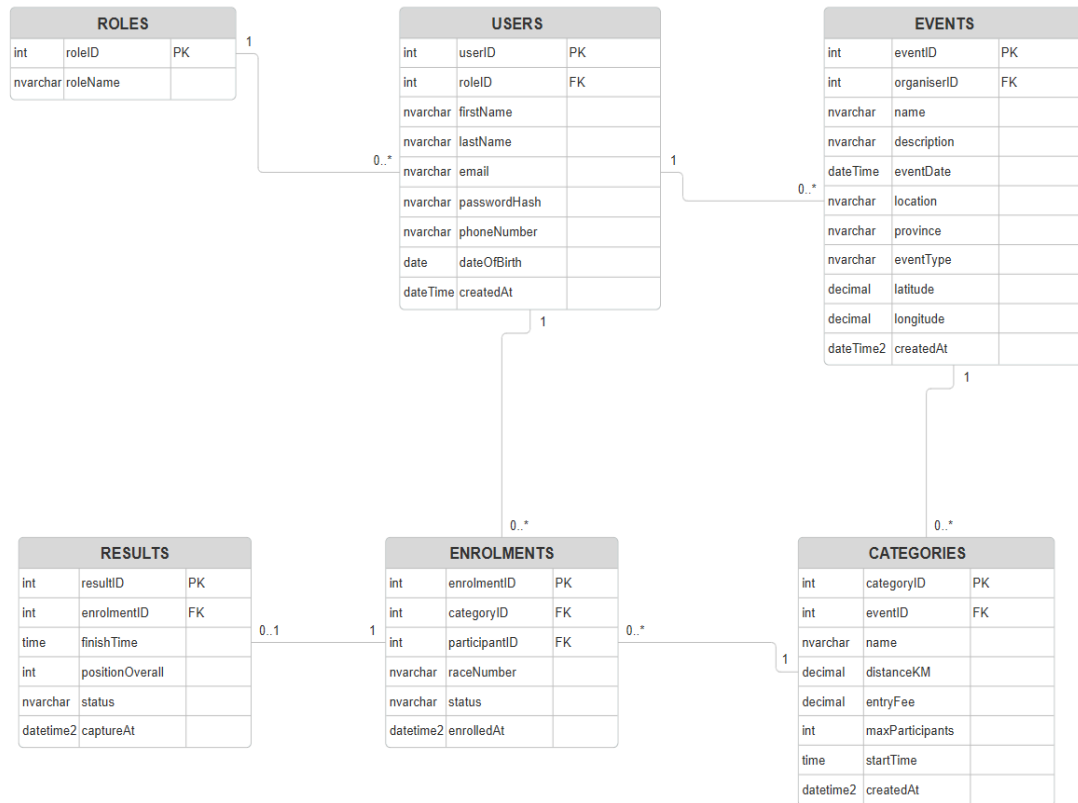
21 September 2026

TABLE OF CONTENTS

SECTION A.....	2
Entity Relationship Diagram (ERD)	2
SECTION B.....	3
API Endpoint Plan.....	3
Authentication (register and login)	3
User Profile	4
Events.....	4
Categories.....	6
Event Enrolments.....	9
Results.....	11
SECTION C	14
SQL Database Script	14
REFERENCES.....	21

SECTION A

Entity Relationship Diagram (ERD)



(Troelsen and Japikse, 2022:817-819), (SmartDraw, 2026)

SECTION B

API Endpoint Plan

Authentication (register and login)

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Creates a new	None	{ firstName, lastName,	201 Created - the new

		account and saves the user with the role the user picks		email, password, phoneNumber, dateOfBirth, roleName }	user id and email 400 Bad Request - a field is missing or the email is not valid 409 Conflict - the email is already used
POST	/api/auth/login	Checks the email and password and gives back a JWT token	None	{ email, password }	200 OK - token, userId and roleName 400 Bad Request - email or password not sent 401 Unauthorized - wrong email or password

(Troelsen and Japikse, 2022:1407-1408), (Microsoft, 2025a), (Mozilla, 2026)

User Profile

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/me	Gets the profile of the user that is logged in, it's used to show the profile page	Any	None	200 OK - the user profile without the password 401 Unauthorized - not logged in

PUT	/api/users/me	Updates the profile details of the user that's logged in	Any	{ firstName, lastName, phoneNumber, dateOfBirth }	200 OK - the updated profile 400 Bad Request - a field is not valid 401 Unauthorized - not logged in
GET	/api/users/{id}	Gets a user. The organiser uses it to see the details of a participant in their event	Organiser	None	200 OK - the user profile 403 Forbidden - the caller is not an organiser 404 Not Found - the user does not exist

Events

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Gets all the events so that participants can browse them	None	None	200 OK - a list of events
GET	/api/events/{id}	Gets one event with its details, and the latitude and longitude that is used for the weather	None	None	200 OK - the event 404 Not Found - the event does not exist

GET	/api/events/my-events	Gets only the events that the user that logged in as the organiser created	Organiser	None	200 OK - a list of the organiser's events 401 Unauthorized - not logged in 403 Forbidden - the caller is not an organiser
POST	/api/events	Makes a new event. The organiser ID is taken from the token, not from the body	Organiser	{ name, description, eventDate, location, province, eventType, latitude, longitude }	201 Created - the new event with its id 400 Bad Request - a field is missing or the date is in the past 403 Forbidden - the caller is not an organiser
PUT	/api/events/{id}	Updates an event. An organiser may only update their own event	Organiser	{ name, description, eventDate, location, province, eventType, latitude, longitude }	200 OK - the updated event 400 Bad Request - a field is not valid 403 Forbidden - the event belongs to another organiser

					404 Not Found - the event does not exist
DELETE	/api/events/{id}	Deletes an event and its categories, as an organiser may only delete their own event	Organiser	None	204 No Content - the event was deleted 403 Forbidden - the event belongs to another organiser 404 Not Found - the event does not exist

Categories

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Gets all the categories of one event so that the participant can pick a distance	None	None	200 OK - a list of categories 404 Not Found - the event does not exist
GET	/api/categories/{id}	Gets one category with its distance, fee and start time	None	None	200 OK - the category 404 Not Found - the category

					does not exist
POST	/api/events/{eventId}/categories	Adds a new category to an event, for example a 10km	Organiser	{ name, distanceKM, entryFee, maxParticipants, startTime }	201 Created - the new category 400 Bad Request - the distance or the fee is not valid 403 Forbidden - the event belongs to another organiser 404 Not Found - the event does not exist 409 Conflict - the event already has a category with that name.
PUT	/api/categories/{id}	Updates a category of an event	Organiser	{ name, distanceKM, entryFee, maxParticipants, startTime }	200 OK - the updated category 400 Bad Request - a field is not valid

					403 Forbidden - the category belongs to another organiser 404 Not Found - the category does not exist
DELETE	/api/categories/{id}	Deletes a category from an event	Organiser	None	204 No Content - the category was deleted 403 Forbidden - the category belongs to another organiser 404 Not Found - the category does not exist 409 Conflict - participants are already entered in this category.

Event Enrolments

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/enrolments	Enters the logged in participant into a category of an event and gives them a race number	Participant	{ categoryid }	201 Created - the enrolment with the race number 400 Bad Request - the category id is missing 403 Forbidden - the caller is not a participant 404 Not Found - the category does not exist 409 Conflict - already entered in this category or the category is full
GET	/api/enrolments/me	Gets all the enrolments of the logged in	Participant	None	200 OK - a list of enrolments with the

		participant so they can see the events they entered in			event and category 401 Unauthorized - not logged in
GET	/api/enrolments/{id}	Gets one enrolment and the participant who owns it or the organiser of the event may see it	Any	None	200 OK - the enrolment 403 Forbidden - the enrolment belongs to another user 404 Not Found - the enrolment does not exist
DELETE	/api/enrolments/{id}	Cancels an enrolment and only the participant who made it may cancel it	Participant	None	204 No Content - the enrolment was cancelled 403 Forbidden - the enrolment belongs to another participant 404 Not Found - the enrolment does not exist

					409 Conflict - the event is already finished
GET	/api/events/{eventId}/enrolments	Gets all the participants that entered an event and the organiser uses it for the start list	Organiser	None	200 OK - a list of enrolments with the participant names 403 Forbidden - the event belongs to another organiser 404 Not Found - the event does not exist

Results

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/results	Captures the result of one enrolment after the race	Organiser	{ enrolmentId, finishTime, positionOverall, status }	201 Created - the new result 400 Bad Request - the finish time is not valid 403 Forbidden - the enrolment is not in the

					organiser's event 404 Not Found - the enrolment does not exist 409 Conflict - the enrolment already has a result
PUT	/api/results/{id}	Updates a result that was captured wrong	Organiser	{ finishTime, positionOverall, status }	200 OK - the updated result 400 Bad Request - a field is not valid 403 Forbidden - the result is not in the organiser's event 404 Not Found - the result does not exist
DELETE	/api/results/{id}	Deletes a result that was captured by mistake	Organiser	None	204 No Content - the result was deleted

					403 Forbidden - the result is not in the organiser's event 404 Not Found - the result does not exist
GET	/api/results/me	Gets all the results of the logged in participant and this would be their personal performance history	Participant	None	200 OK - a list of results with the event, category and finish time 401 Unauthorized - not logged in
GET	/api/events/{eventId}/results	Gets the result list of an event so that anybody can see who won	None	None	200 OK - a list of results in position order 404 Not Found - the event does not exist

SECTION C

SQL Database Script

```
IF DB_ID('RaceDayDB') IS NULL
BEGIN
    CREATE DATABASE RaceDayDB;
END
GO
```

```
USE RaceDayDB;
GO
```

```
DROP TABLE IF EXISTS Results;
DROP TABLE IF EXISTS Enrolments;
DROP TABLE IF EXISTS Categories;
DROP TABLE IF EXISTS Events;
DROP TABLE IF EXISTS Users;
DROP TABLE IF EXISTS Roles;
GO
```

```
CREATE TABLE Roles (
    roleID INT IDENTITY(1,1) PRIMARY KEY,
    roleName NVARCHAR(20) NOT NULL,

    CONSTRAINT UQ_Roles_roleName UNIQUE (roleName),
    CONSTRAINT CK_Roles_roleName CHECK (roleName IN ('Organiser', 'Participant'))
);
GO
```

```
CREATE TABLE Users (
    userID INT IDENTITY(1,1) PRIMARY KEY,
```

```

roleID    INT NOT NULL,

firstName NVARCHAR(100) NOT NULL,

lastName  NVARCHAR(100) NOT NULL,

email     NVARCHAR(150) NOT NULL,

passwordHash NVARCHAR(255) NOT NULL,

phoneNumber NVARCHAR(20) NULL,

dateOfBirth DATE NULL,

createdAt  DATETIME2 NOT NULL DEFAULT GETDATE(),


CONSTRAINT UQ_Users_email UNIQUE (email),

CONSTRAINT FK_Users_Role FOREIGN KEY (roleID) REFERENCES Roles(roleID)

);

GO

```

```

CREATE TABLE Events (

    eventID    INT IDENTITY(1,1) PRIMARY KEY,

    organiserID INT NOT NULL,

    name       NVARCHAR(150) NOT NULL,

    description NVARCHAR(MAX) NULL,

    eventDate  DATETIME2 NOT NULL,

    location   NVARCHAR(150) NOT NULL,

    province   NVARCHAR(100) NOT NULL,

    eventType  NVARCHAR(20) NOT NULL,

    latitude   DECIMAL(9,6) NULL,

    longitude  DECIMAL(9,6) NULL,

    createdAt  DATETIME2 NOT NULL DEFAULT GETDATE(),

```

```
CONSTRAINT FK_Events_Organiser FOREIGN KEY (organiserID) REFERENCES
Users(userID),

CONSTRAINT CK_Events_eventType CHECK (eventType IN ('Running', 'Walking',
'Cycling'))

);

GO
```

```
CREATE TABLE Categories (

    categoryID    INT IDENTITY(1,1) PRIMARY KEY,

    eventID       INT NOT NULL,

    name          NVARCHAR(100) NOT NULL,

    distanceKM     DECIMAL(6,2) NOT NULL,

    entryFee       DECIMAL(10,2) NOT NULL DEFAULT 0,

    maxParticipants INT NULL,

    startTime      TIME(0) NULL,

    createdAt      DATETIME2 NOT NULL DEFAULT GETDATE(),
```

```
CONSTRAINT FK_Categories_Event FOREIGN KEY (eventID) REFERENCES
Events(eventID) ON DELETE CASCADE,

CONSTRAINT UQ_Categories_EventName UNIQUE (eventID, name),

CONSTRAINT CK_Categories_distanceKM CHECK (distanceKM > 0),

CONSTRAINT CK_Categories_entryFee CHECK (entryFee >= 0)

);

GO
```

```
CREATE TABLE Enrolments (

    enrolmentID    INT IDENTITY(1,1) PRIMARY KEY,

    categoryID     INT NOT NULL,

    participantID   INT NOT NULL,
```

```

raceNumber    NVARCHAR(10) NOT NULL,

status        NVARCHAR(20) NOT NULL DEFAULT 'Confirmed',

enrolledAt    DATETIME2 NOT NULL DEFAULT GETDATE(),


CONSTRAINT FK_Enrolments_Category FOREIGN KEY (categoryID) REFERENCES
Categories(categoryID) ON DELETE CASCADE,

CONSTRAINT FK_Enrolments_Participant FOREIGN KEY (participantID) REFERENCES
Users(userID),

CONSTRAINT UQ_Enrolments_raceNumber UNIQUE (raceNumber),

CONSTRAINT UQ_Enrolments_CategoryParticipant UNIQUE (categoryID,
participantID),

CONSTRAINT CK_Enrolments_status CHECK (status IN ('Confirmed', 'Cancelled'))

);

GO

```

```

CREATE TABLE Results (

resultID      INT IDENTITY(1,1) PRIMARY KEY,

enrolmentID   INT NOT NULL,

finishTime    TIME(0) NULL,

positionOverall INT NULL,

status        NVARCHAR(20) NOT NULL DEFAULT 'Finished',

capturedAt    DATETIME2 NOT NULL DEFAULT GETDATE(),


CONSTRAINT FK_Results_Enrolment FOREIGN KEY (enrolmentID) REFERENCES
Enrolments(enrolmentID) ON DELETE CASCADE,

CONSTRAINT UQ_Results_enrolment UNIQUE (enrolmentID),

CONSTRAINT CK_Results_status CHECK (status IN ('Finished', 'DNF', 'DNS')),

CONSTRAINT CK_Results_position CHECK (positionOverall IS NULL OR
positionOverall > 0)

```

);

GO

INSERT INTO Roles (roleName) VALUES

('Organiser'),

('Participant');

GO

INSERT INTO Users (roleID, firstName, lastName, email, passwordHash,
phoneNumber, dateOfBirth) VALUES

(1, N'Thabo', N'Nkosi', N'thabo.nkosi@raceday.co.za', N'HASH_Organiser01',
N'0821234567', '1985-03-14'),

(1, N'Naledi', N'Mokoena', N'naledi.mokoena@raceday.co.za', N'HASH_Organiser02',
N'0839876543', '1990-07-22'),

(2, N'Sipho', N'Khumalo', N'sipho.khumalo@gmail.com', N'HASH_Participant01',
N'0712223333', '1996-11-02'),

(2, N'Chantal', N'Pillay', N'chantal.pillay@gmail.com', N'HASH_Participant02',
N'0734445555', '1993-05-30'),

(2, N'Lerato', N'Dlamini', N'lerato.dlamini@gmail.com', N'HASH_Participant03',
N'0766667777', '2000-01-18');

GO

INSERT INTO Events (organiserID, name, description, eventDate, location, province,
eventType, latitude, longitude) VALUES

(1, N'Durban Beachfront 10km Challenge',

N'A flat and fast race along the Golden Mile promenade. The race starts at uShaka
Marine World.',

'2026-08-16 06:00:00', N'Durban', N'KwaZulu-Natal', N'Running', -29.867300,
31.045400),

(1, N'Pietermaritzburg Hill Half Marathon',

N'A hard half marathon through the hills of Pietermaritzburg. Water tables every 3km.',

'2026-11-08 05:30:00', N'Pietermaritzburg', N'KwaZulu-Natal', N'Running', -29.601100, 30.379400),

(2, N'Cape Winelands Cycle Classic',

N'A cycling event through the wine farms of Paarl. Helmets are compulsory for all riders.',

'2026-12-06 07:00:00', N'Paarl', N'Western Cape', N'Cycling', -33.731700, 18.962100);

GO

INSERT INTO Categories (eventID, name, distanceKM, entryFee, maxParticipants, startTime) VALUES

(1, N'10km Run', 10.00, 120.00, 800, '06:00'),

(1, N'5km Fun Run and Walk', 5.00, 60.00, 500, '06:30'),

(2, N'21.1km Half Marathon', 21.10, 180.00, 1000, '05:30'),

(2, N'10km Run', 10.00, 100.00, 600, '06:00'),

(3, N'50km Cycle', 50.00, 350.00, 1200, '07:00'),

(3, N'25km Cycle', 25.00, 200.00, 900, '07:30');

GO

INSERT INTO Enrolments (categoryID, participantID, raceNumber, status) VALUES

(1, 3, N'D1001', N'Confirmed'),

(2, 4, N'D1002', N'Confirmed'),

(1, 5, N'D1003', N'Confirmed'),

(3, 3, N'P2001', N'Confirmed'),

(6, 4, N'C3001', N'Confirmed');

GO

```
INSERT INTO Results (enrolmentID, finishTime, positionOverall, status) VALUES
(1, '00:47:32', 1, N'Finished'),
(2, '00:31:05', 1, N'Finished'),
(3, '00:52:18', 2, N'Finished');
GO
```

```
SELECT
    e.name AS EventName,
    e.eventDate,
    e.location,
    c.name AS CategoryName,
    c.distanceKM,
    c.entryFee
FROM Events e
INNER JOIN Categories c ON c.eventID = e.eventID
ORDER BY e.eventDate, c.distanceKM;
```

```
SELECT
    u.firstName + ' ' + u.lastName AS Participant,
    e.name AS EventName,
    c.name AS CategoryName,
    en.raceNumber,
    r.finishTime,
    r.positionOverall
FROM Enrolments en
INNER JOIN Users u ON u.userID = en.participantID
INNER JOIN Categories c ON c.categoryID = en.categoryID
```

INNER JOIN Events e ON e.eventID = c.eventID

LEFT JOIN Results r ON r.enrolmentID = en.enrolmentID

ORDER BY e.eventDate, c.name, r.positionOverall;

GO /* Troelsen and Japikse, 2022:817-819; Microsoft, 2024; Microsoft, 2025b;
W3Schools, 2026 */

REFERENCES

- Troelsen, A. and Japikse, P. 2022. Pro C# 10 with .NET 6: foundational principles and practices in programming (11th ed.). Berkeley, CA: Apress.
- SmartDraw. 2026. Easy Floor Plans and Other Layouts. Available at: <https://app.smartdraw.com/> [Accessed: 21 September 2026]
- Microsoft. 2025a. Best practices for RESTful web API design. Available at: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design> [Accessed: 21 September 2026]
- Mozilla. 2026. HTTP response status codes. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status> [Accessed: 21 September 2026]
- Microsoft. 2025b. Create foreign key relationships. Available at: <https://learn.microsoft.com/en-us/sql/relational-databases/tables/create-foreign-key-relationships> [Accessed: 21 September 2026]
- Microsoft. 2026. Data types (Transact-SQL). Available at: <https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-types-transact-sql> [Accessed: 21 September 2026]
- W3Schools. 2026. SQL Constraints. Available at: https://www.w3schools.com/sql/sql_constraints.asp [Accessed: 21 September 2026]